



DATA STRUCTURES

CONTENTS



14. Priority Queue and Applications

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of an important ADT – the Queue ADT. In this module we will discuss an important type of Queue ADT – the Priority Queue ADT. We will also discuss some important applications of Priority Queue ADT.

Learning Objectives

The learning objectives of the introductory module are as follows:

- To explain the concept of Priority Queue ADT
- To outline the various operations of Priority Queue
- To explain the implementation of Priority Queue using Unsorted and Sorted Sequence
- To discuss the applications of Priority Queue

14.1 Queues and Priority Queues

As we already seen the general operations on queues include Enqueue(e) where the new element e is inserted to the rear of the queue and Dequeue() where the element at the front of the queue which has been entered first is removed first.

Now we discuss a slight variation to the queue data structure where the priority should not be implicit using the time of entry as is the case in of normal queues but the priority is *explicit*, using an ordering. In other words we now have a special form of queue from which items are removed according to their designated priority and not the order in which they entered. Figure 14.1 shows that items are entered into the queue in sequential order but will be removed in the order of seniority that is items #3, #1, #4, #2 will be the order in which deletion will take place.

There are many situations where priority associated with the queue plays an important role. One such example is a printer queue where the number of pages can be used as priority to ensure that longer jobs do not dominate the printer. Another example discrete event simulation is where events are queued with the simulation time is used as priority. The next example is network routing where the priority queue is used to store packets where packets with stricter quality-of-service are given higher priority.

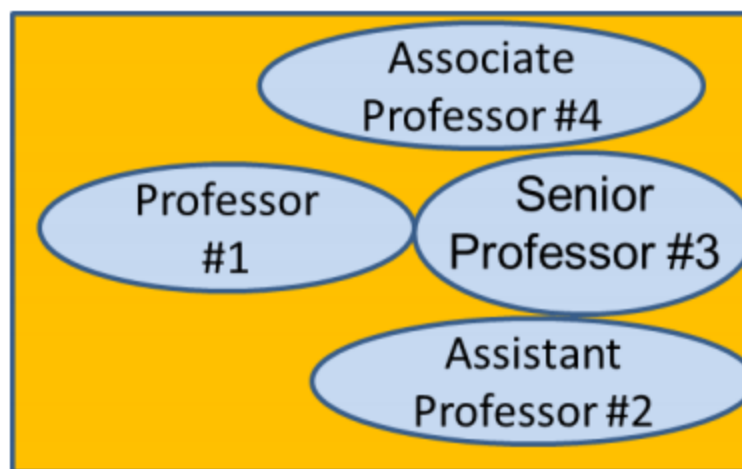


Figure 14.1 Items and their Priority

14.2 Priority Queue ADT

A Priority Queue is an Abstract Data Type like Stacks, Queues, etc. However in a Priority Queue each item (element) in addition to having data will also have an associated priority. The operation that must be very efficient in a priority queue is finding the item with highest priority (usually the one with the maximum or minimum key value). A priority queue is a collection of zero or more elements where each element has a **priority** associated with it. Unlike the FIFO queues, the **order** of deletion from a priority queue (e.g. who gets served next) is determined by the priority of the element. Elements are deleted by increasing or decreasing order of priority rather than by the order in which they arrived in the queue. We need to ensure that the task with the highest priority task is at the front of the queue. Keys in a priority queue can be arbitrary objects on which an order is defined. Note that two distinct entries in a priority queue can have the same priority.

14.2.1 Basis of Priority Queue

A priority queue ranks its elements by key with a total order relation. Every element has its own key and keys are not necessarily unique. Now let us discuss the mathematical concept of total order relation $\leq$. The following properties hold good:

- Reflexive property: The Reflexive Property states that for every number x , $x \leq x$
- Antisymmetric property: A relation R on a set S is called antisymmetric if, whenever $x R y$ and $y R x$ are both true, then $x=y$. In our case on the set Z of integer numbers $x \leq y$ and $y \leq x$ then $x = y$.
- Transitive property: The Transitive Property states that for all numbers x , y , and z , $x \leq y$ and $y \leq z$ then $x \leq z$

The above conditions are also applicable to partial order relation. However in the case of total order relation it has an additional condition known as the comparability condition.

- Comparability condition states that for any $x, y \in S$, either $x \leq y$ or $y \leq x$.

Now let us discuss the various operations performed on priority queues. Typical operations include:

- Finding an element,
- insert a new element,

- delete an element, etc.

Two kinds of (Min, Max) priority queues exist. In a **Min priority queue**, find/delete operation finds/deletes the element with minimum priority while in a **Max priority queue**, find/delete operation finds/deletes the element with maximum priority. The difference between Max priority queue specification and Min priority queue specification would be the way in which the delete operation is carried out. Can you think of other examples in our daily lives that utilize the priority queue concept?

14.2.2 Entry ADT

We have redefined the element in the priority queue as consisting of a key-value pair. An **entry** in a priority queue is simply this key-value pair. Priority queues store entries to allow for efficient insertion and removal based on keys. Some of the operations associated with entries include operations to access either the key component of entry or the value component of entry.

- **key()**: returns the key for this entry
- **value()**: returns the value associated with this entry

14.3 Operations of Priority Queue ADT

As already discussed an item is a pair (key, element). The main methods of the Priority Queue ADT are

- **insertItem(k, o)** -inserts an item with key k and element o
- **removeMin()** -removes the item with smallest key and returns its element

size(): Returns the number of elements in priority queue P

isEmpty(): Test whether P is empty

insertItem(k,e): Insert a new element e with key k into P

minElement(): Return (but don't remove) an element of P with smallest key; an error occurs if P is empty.

minKey(): Return the smallest key in P; an error occurs if P is empty

removeMin(): Remove from P and return an element with the smallest key; an

error condition occurs if P is empty.

14.3.1 Comparator

A comparator compares two objects according to a given total order relation. Normally priority queue uses an auxiliary comparator. The comparator is external to the keys being compared.

The primary function is the comparison of the :

compare(a, b): Returns an integer i such that

- $i < 0$ if $a < b$,
- $i = 0$ if $a = b$, and
- $i > 0$ if $a > b$; an error occurs if a and b cannot be compared.

When the priority queue needs to compare two keys, it uses the comparator to compare the two keys. Some of the other operations that can be defined for the comparator include:

-isLessThan(a, b) – This is a boolean operator that takes two inputs a , b and returns true if $a < b$ and false otherwise.

-isLessThanOrEqualTo(a,b) – This is a boolean operator that takes two inputs a , b and returns true if $a \leq b$ and false otherwise.

-isEqualTo(a, b) – This is a boolean operator that takes two inputs a , b and returns true only if $a = b$ and false otherwise.

-isGreaterThan(a,b) – This is a boolean operator that takes two inputs a , b and returns true only if $a > b$ and false otherwise.

-isGreaterThanOrEqualTo(a,b) – This is a boolean operator that takes two inputs a , b and returns true only if $a \geq b$ and false otherwise.

14.4 Implementation with an Unsorted Sequence

Next we will discuss some of the ways in which the priority queue can be implemented. The first implementation we will discuss is the implementation of priority queue using an unsorted sequence S . The elements of the sequence S are made up of two components namely k , the key, and e , the element. Now since we

are using an unsorted sequence the implementation of `insertItem()` can be carried out by using `insertLast()`. In the example shown in Figure 14.2 the element with key 6 is added as the last element. This takes $O(1)$ time. However, because we always insert at the end, irrespective of the key value, our sequence is not ordered.

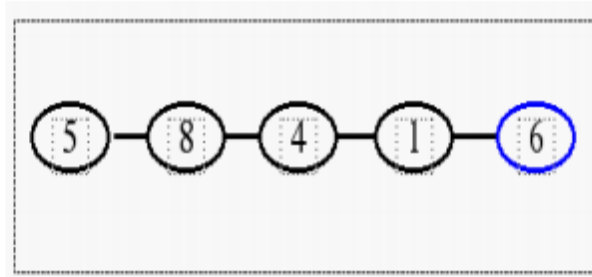


Figure 14.2 Implementation with an Unsorted Sequence

Thus, for methods such as `minElement()`, `minKey()`, and `removeMin()`, all the elements of S have to be examined (Figure 14.3). Therefore the worst case time complexity for these methods is $O(n)$. The implementation with an unsorted sequence can be carried out using an unsorted list for the sequence. As discussed previously insert takes $O(1)$ time since in case of an appropriately defined list ADT we can insert the item at the beginning or end of the sequence. However `removeMin` and `min` take $O(n)$ time since we have to traverse the entire sequence to find the smallest key

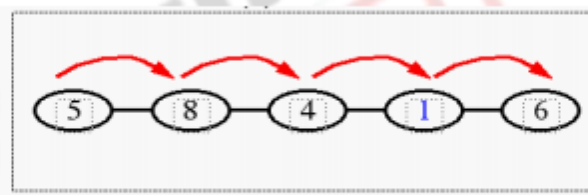


Figure 14.3 Examining all elements in an Unsorted Sequence

14.5 Implementation with Sorted Sequence

Another implementation of the priority queue uses a sequence S , sorted by increasing keys. In this case the methods `minElement()`, `minKey()`, and `removeMin()` take $O(1)$ time. However, in the case of `insertItem()`, we are inserting into a sorted sequence and hence need to look through the entire sequence to find the correct position for the item to be inserted and in the worst case this may require looking through the entire sequence and hence `insertItem()` runs in $O(n)$ time. The implementation with a sorted sequence can be carried out using a sorted list for the sequence. As discussed previously insert takes $O(n)$ time since we have to find the place where to insert the item. However `removeMin` and `min` take $O(1)$ time, since the smallest key is at the beginning of the list.

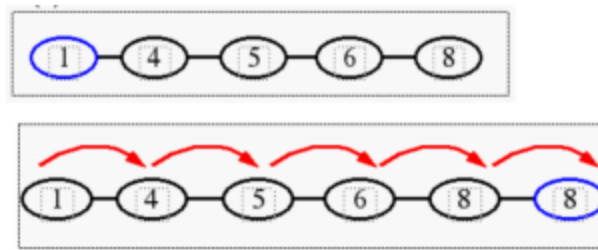


Figure 14.4 Implementation with Sorted Sequence

14.5 Applications of Priority Queues

As we have already discussed, priority queue is a very important data structure, used in many different types of contexts and applications. Table 14.1 lists the different type of application and examples of each type of application. One of the common types of application of priority queues is in simulation. A simple example is the simulation of a hospital waiting room queue where patients might be prioritized based on the severity of their need. A common form of simulation is termed a “discrete, event-driven simulation”. In this application the simulation is defined in term of a series of “events”. In this case an event is simply a representation of some action that occurs at a given time. The priority queue maintains a collection of events, and the event with highest priority will be the event with lowest time; that is, the event that will occur next. Giving priority to events with lowest time is a common requirement in many applications.

Another type of application of priority queues is in data compression. An example is the use of priority queues in Huffman's encoding. In this scenario each character along with its count of occurrences is placed into a priority queue where the character with maximum frequency of occurrence is at the front of the queue. This means that the encoding of this character will take place before other characters are encoded.

Priority queue finds application in graph searching algorithms where the items are ordered based on the length or cost of the edge or path between two nodes of the graph. These queues are also used when we need to allocate resources based on memory requirement, time to finish a job, etc.. in the case of operating systems. Discrete optimization problems like scheduling, bin packing etc. where the optimization is based on some constraints also uses the priority queue.

Type of Application	Applications
Event-driven simulation	Customers in a line, colliding particles
Data compression	Huffman codes
Graph searching	Dijkstra's algorithm, Prim's algorithm
Operating systems.	load balancing, interrupt handling
Discrete optimization.	bin packing, scheduling

Table 14.1 Applications of Priority Queues

14.6 Priority Queue Sorting

A Priority Queue P can be used for sorting a sequence S by carrying out two operations:

- **Inserting** the elements of S into an initially empty priority queue P with a series of **insert** (k,v) operations where each element is a key-value pair

- **Removing** the elements from P in increasing order and putting them back into S with a series of **RemoveMin()** operations

The running time of this sorting method depends on the priority queue implementation. Figure 14.6(a) and (b) shows the insertion of 4 elements of the sequence S into the priority queue P. Figure 14.6 (c) – (f) shows the removals of the minimum element from P and the insertions back into the sequence S.

Given Sequence S and Priority Queue P

inserting the elements of S into P with a series of **P.insert(k,v)** operations

Operation	Sequence S	Priority Queue P
P.Insert(A,1)	A	1
P.Insert(X,7)	A X	1 7
P.Insert(C,2)	A X C	1 7 2
P.Insert(L,5)	A X C L	1 7 2 5

(a)

Inserting elements (Value, Key) of the Sequence into the Priority Queue one by one

P.Insert (T,6)	A	X	C	L	T				
	1	7	2	5	6				
P.Insert (D,3)	A	X	C	L	T	D			
	1	7	2	5	6	3			
P.Insert (K,4)	A	X	C	L	T	D	K		
	1	7	2	5	6	3	4		

(b)

removing the elements from P in increasing order and putting them back into S with a series of **removeMin()** operations

P.RemoveMin()

X	C	L	T	D	K				
7	2	5	6	3	4				

S.InsertLast(X)

A									
1									

(c)

P.RemoveMin()

X	L	T	D	K					
7	5	6	3	4					

S.InsertLast(X)

A	C								
1	2								

P.RemoveMin()

X	T	L	K						
7	6	5	4						

S.InsertLast(X)

A	C	D							
1	2	3							

(d)

P.RemoveMin()

X	T	L							
7	6	5							

S.InsertLast(X)

A	C	D	K						
1	2	3	4						

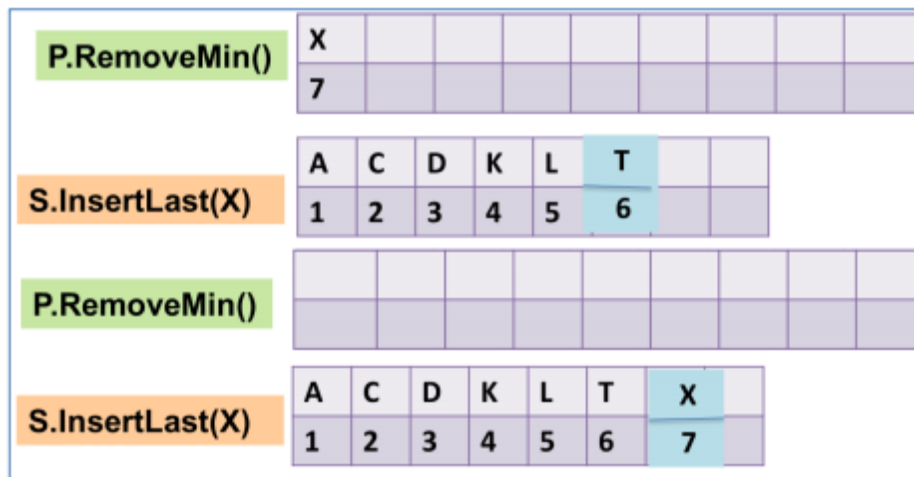
P.RemoveMin()

X	T								
7	6								

S.InsertLast(X)

A	C	D	K	L					
1	2	3	4	5					

(e)



(f)

Figure 14.5 Use of Priority Queue for Sorting

14.6.1 Selection-Sort

In selection-sort the priority queue is implemented using an unsorted sequence. Analyzing the running time of Selection-sort we know that there are two phases (Figure 14.6) namely inserting the elements into the priority queue with n insert operations which takes $O(n)$ time and removing the elements in sorted order from the priority queue with n removeMin operations that takes time proportional to $1 + 2 + \dots + n$. Therefore selection-sort runs in $O(n^2)$ time.

	Sequence S	Priority Queue P
Input	(4,8,2,9,3,1,7,5)	()
Phase I	(8,2,9,3,1,7,5)	(4)
	(2,9,3,1,7,5)	(4,8)
	(9,3,1,7,5)	(4,8,2)
		
		
	()	(4,8,2,9,3,1,7,5)
Phase II	(1)	(4,8,2,9,3,7,5)
	(1,2)	(4,8,9,3,7,5)
	(1,2,3)	(4,8,9,7,5)
	(1,2,3,4)	(8,9,7,5)
	(1,2,3,4,5)	(8,9,7)
	(1,2,3,4,5,7)	(8,9)
	(1,2,3,4,5,7,8)	(9)
	(1,2,3,4,5,7,8,9)	()

Figure 14.6 Priority Queue for Selection Sorting

14.6.2 Insertion-Sort

Insertion-sort is the variation of priority Queue sort where the priority queue is implemented with a sorted sequence. Analyzing the running time of insertion-sort we know that there are two phases namely inserting the elements into the priority queue with n insert operations takes time proportional to $1 + 2 + \dots + n$ **and** removing the elements in sorted order from the priority queue with a series of n removeMin operations that takes $O(n)$ time. Therefore the running time of insertion-sort is $O(n^2)$.

14.6.3 In-place Sorting

Instead of using an external data structure, we can implement selection-sort and insertion-sort in-place where a portion of the input sequences itself serves as the priority queue. For in-place insertion-sort, we keep sorted the initial portion of the

sequence and use swaps instead of modifying the sequence. In the example given in Figure 14.6 we interchange values that are out of place starting with 5 and 4 where the beginning of two positions of the input sequence itself is the priority queue. This process continues and each time one more position is occupied by the elements checked up to that point example 2, 4 and 5 occupying positions 1, 2 and 3 respectively 4th row. This is a result of two swaps at row 3, between 2 & 5 and 2 & 4.

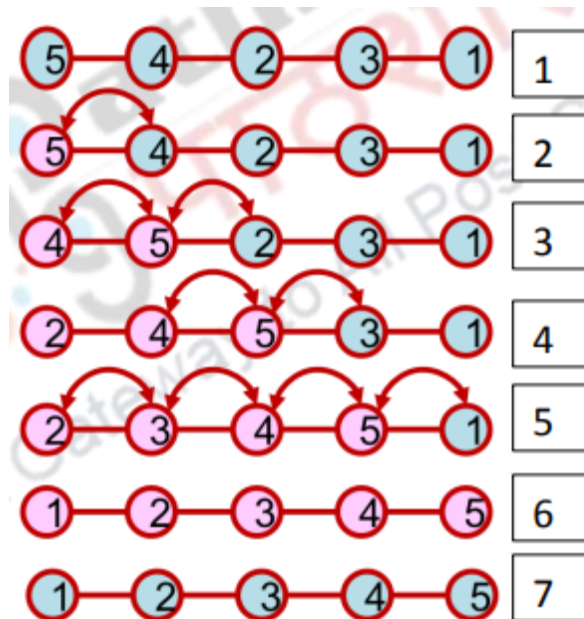


Figure 14.6 Priority Queue for In-Place Sorting

Summary

- Explained the concept of Priority Queue and Priority Queue ADT
- Outlined the various operations of Priority Queues
- Explained the implementation of Priority Queue using Unsorted and Sorted Sequence
- Discussed the various applications of Priority Queue especially for Sorting

you can view video on Priority Queue and Applications



Web Links
cs.calvin.edu/curriculum/cs/212/hplantin/lectures/ch02.4.ppt
www.cs.nyu.edu/~melamed/courses/102/lectures/PriorityQueues.ppt
ftp.csd.uwo.ca/courses/CS1027b/notes/CS1027-009-Queues-W12.ppt
www.cs.princeton.edu/~wayne/cs423/lectures/demo-bfs.ppt
www.uwinnipeg.ca/~ychen2/AlgorithmNotes/class14.ppt
itu.dk/people/pagh/ads11/07-PriorityQueues.pdf
http://www.postech.ac.kr/class/csed233-01/lec_notes/2003lecture11.ppt
http://girdhargopalbansal.blogspot.in/2013/07/priority-queues-insertion-sort.html
www.site.uottawa.ca/~flocchin/CSI2114-04/SLIDES/06-6pp.pdf
Supporting & Reference Materials
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Adam Drozdek, "Data Structures and Algorithms in C++", Cengage; 4th edition, 2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014